Advanced Big Data Analytics
AUCA

Group Members:

S/N	NAMES	STUDENT ID
1	**VICTOR DUSENGIMANA**	**101372**
2	**UMWIZA ODILE**	**101365**
3	**IGIRUKWAYO ILDEPHONSE**	**101404**
4	**MUKIZA ALAIN CHRISTIAN**	**101370**

SMARTENERGYGRIDANALYTICS Complete Research and Technical Report

From MQTT ingestion to scalable time-series analytics, anomaly detection and forecasting

Advanced Big Data Analytics
AUCA

Contents

.... 4

Executive Summary at a Glance 5

1. Executive Summary 6

Question 1. Problem Statement, Objectives and Dataset Scope 7

1.1 Problem Statement, Objectives and Research Questions 7

1.2 Dataset Scope, Schema and Time Semantics 7

Question 2. System Architecture and MQTT/EMQX Ingestion 8

2.1 System Architecture and Technology Environment 8

2.2 MQTT and EMQX Ingestion Pilot 8

Question 3. Database Design and Data Quality 9

3.1 Database Design and TimescaleDB Hypertables 9

3.2 Data Quality, Deduplication and Reconciliation 9

Question 4. Chunk-Interval Benchmarking 10

4.1 Benchmark Methodology and Query Definitions 10

4.2 Chunk-Interval Benchmark Results 11

4.3 Chunk Storage and Operational Trade-offs 12

Question 5. Compression Design and Performance 12

5.1 Compression Design and Storage Results 12

5.2 Compression Query Performance 13

Question 6. Energy Analytics, Aggregates and Dashboards 14

6.1 Regional and Customer-Type Energy Analytics 14

6.2 Weekly Energy Trend and Network Demand Peaks 14

6.3 Regional Data Completeness 15

6.4 Continuous Aggregates and Refresh Policies 15

6.5 Materialized versus Real-Time Aggregate Freshness 16

6.6 Grafana Dashboard Validation 16

Question 7. Anomaly Detection 16

7.1 Anomaly Detection Method 16

7.2 Anomaly Detection Results and Interpretation 16

Question 8. Forecasting and Error Analysis 17

8.1 Forecasting Objective and Dataset 17

8.2 Forecast Features and Leakage Controls 18

8.3 Previous-Day Baseline and Random Forest Model 18

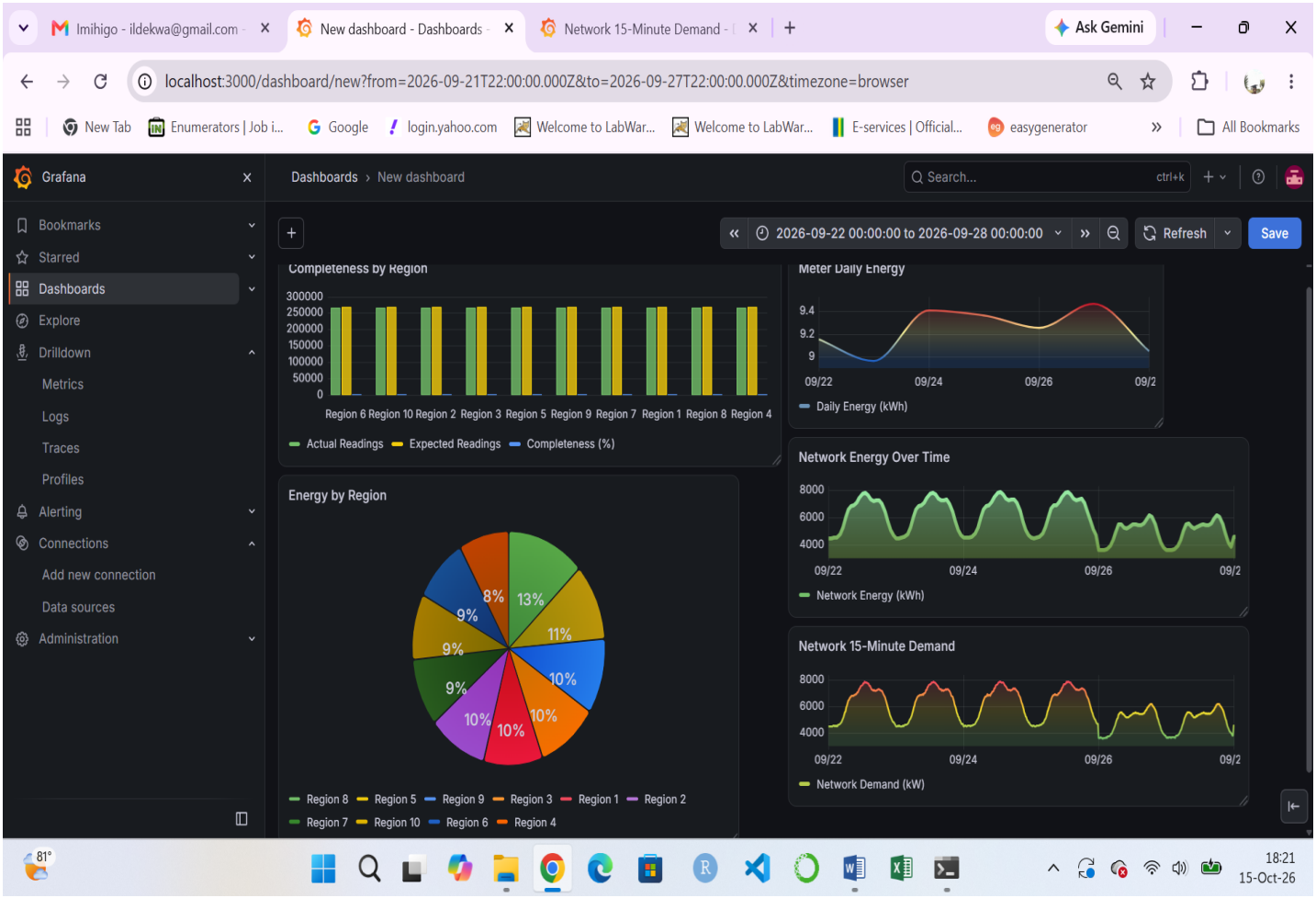
8.4 Regional Forecast Errors 18

8.5 Forecast Interpretation and Error Analysis 19

8.6 Forecasting Recommendation 19

Question 9. Recommendations, Limitations and Reproducibility 19
9.1 Operational Recommendations and Next Steps 19
9.2 Limitations and Validity Considerations 19
9.3 Reproducibility and Evidence Checklist 20
9.4 Conclusion 21
Additional Research Material and Appendices and Analysis Chartd 24
1. Completeness by Region 24
Supplementary Note A — Advanced Model Comparisons 26
Supplementary Note B — Distributed Big Data Architecture 26
Supplementary Conclusion 26
Supplementary Reproducibility and Evidence Checklist 27
a) Environment and ingestion 27
b) Data generation and quality 27
c) Database design and benchmarking 27
d) Analytics and anomaly detection 27
e) Continuous aggregates and dashboards 27
f) Forecasting 28
h) Appendices: code examples and analytical result references 28
Appendix A — Database connection and initial validation 28
Appendix B — Validate the cleaned dataset 29
Appendix C — Verify energy reconciliation 29
Appendix D — Calculate query-performance improvement 29
Appendix E — Calculate forecasting metrics 30
Appendix F — Final results reference 30
Study results at a glance 30
Final research statement 31
G. Screen Short: 31

Project Dashboard



Executive Summary at a Glance

This project evaluates a smart-meter analytics pipeline using an MQTT/EMQX ingestion pilot, PostgreSQL with TimescaleDB for time-series storage, continuous aggregates for repeated reporting, Grafana for dashboard validation, rule-based anomaly detection, and a Random Forest model for next-day hourly regional energy forecasting. The full historical load was inserted directly into the database for efficiency; the MQTT route was validated separately with a 2,000-message pilot.,

Measure	Reported result
Registered meters	4,000 (IDs 1000000000–1000003999)
Sampling interval	15 minutes; 96 expected readings per meter per day
Planned records	10,752,000
Raw records loaded	10,696,848
Unique cleaned readings	10,643,735
Duplicate extras removed	53,113
Expected readings missing	108,265
Delayed unique readings	213,090
Chunk benchmark winners	Q1/Q2: 3 hours; Q3/Q5: 1 week; Q4: 1 day
Compression storage	1,749 MB baseline; 524 MB time-ordered; 590 MB meter-segmented
Anomaly detection	40/40 injected event intervals detected; 0 observed false-flagged readings
Forecast test MAE	42.708281 kWh baseline; 5.054452 kWh Random Forest
Forecast test RMSE	79.402867 kWh baseline; 6.787988 kWh Random Forest

The benchmark results do not support a universal best chunk interval. A 1-day interval is a balanced operational baseline, while workload-specific layouts can be selected for time-focused or meter-focused queries. Forecast results are promising but remain bounded by a synthetic dataset and a seven-day test window.

1. Executive Summary

Project problem

A smart grid produces high-volume time-series readings from many meters.

Operational systems need timely data; analysts need reliable historical data.

Duplicate, missing, and delayed readings can distort totals and operational decisions.

Large tables require deliberate storage, indexing, aggregation, and query design.

The project tests a practical pipeline and evaluates it using measured results.

Objectives and research questions

Ingest meter readings through an MQTT pilot and persist them in PostgreSQL/TimescaleDB.

Load the full historical dataset and verify its scale and quality.

Compare 1-day, 3-hour, and 1-week hypertable chunk intervals using five queries.

Evaluate compression layouts, continuous aggregates, dashboard equivalence, anomaly detection, and forecasting.

Make operational recommendations grounded in measurements rather than assumptions.

Dataset at a glance

- Registered meters: 4,000
- Meter IDs 1000000000–1000003999

Sampling interval:

- 15 minutes
- 96 planned readings per meter per day

Time window:

- 28 days, 1 Sep 00:00 to 29 Sep 00:00 exclusive, CAT

Planned records

- 10,752,000
- 4,000 × 28 × 96

Time semantics and measurement meaning

event_time: the time interval the measurement describes.

received_time: the simulated time the reading arrives; it is not the database insertion time.

power_kw measures power; energy_kwh is interval energy and is approximately power_kw × 0.25 hours.

All reporting uses CAT / Africa-Kigali consistently.

Missing intervals remain gaps; they must not be converted to zero energy.

Question 1. Problem Statement, Objectives and Dataset Scope

1.1 Problem Statement, Objectives and Research Questions

Project problem

A smart grid produces high-volume time-series readings from many meters.

Operational systems need timely data; analysts need reliable historical data.

Duplicate, missing, and delayed readings can distort totals and operational decisions.

Large tables require deliberate storage, indexing, aggregation, and query design.

The project tests a practical pipeline and evaluates it using measured results.

Objectives:

1. Ingest meter readings through an MQTT pilot and persist them in PostgreSQL/TimescaleDB.

2. Load the full historical dataset and verify its scale and quality.

3. Compare 1-day, 3-hour, and 1-week hypertable chunk intervals using five queries.

4. Evaluate compression layouts, continuous aggregates, dashboard equivalence, anomaly detection, and forecasting.

5. Make operational recommendations grounded in measurements rather than assumptions.

1.2 Dataset Scope, Schema and Time Semantics

Dataset at a glance

- Registered meters: 4,000
- Meter IDs 1000000000–1000003999

Sampling interval:

- 15 minutes
- 96 planned readings per meter per day

Time window:

- 28 days, 1 Sep 00:00 to 29 Sep 00:00 exclusive, CAT

Planned records

- 10,752,000
- 4,000 × 28 × 96

Time semantics and measurement meaning:

- event_time: the time interval the measurement describes.
- received_time: the simulated time the reading arrives; it is not the database insertion time.

power_kw measures power; energy_kwh is interval energy and is approximately power_kw × 0.25 hours.

All reporting uses CAT / Africa-Kigali consistently.

Missing intervals remain gaps; they must not be converted to zero energy.

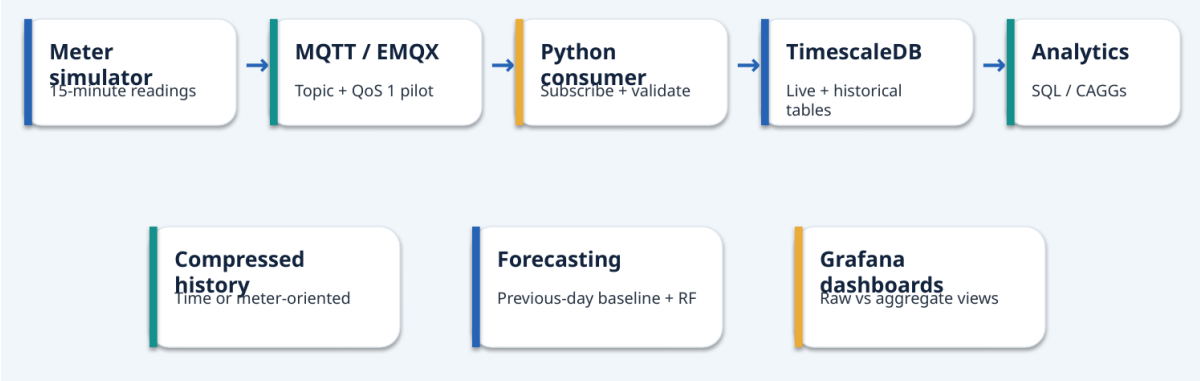
Database design

- meters store meter metadata: meter_id, region, customer_type.
- energy_live is a normal PostgreSQL table for the pilot/operational path.
- energy_readings is the historical hypertable partitioned by event_time.
- The reading table has exactly eight fields: meter_id, event_time, received_time, power_kw, voltage_v, current_a, frequency_hz, energy_kwh.

The original generator was left unchanged.

Question 2. System Architecture and MQTT/EMQX Ingestion

2.1 System Architecture and Technology Environment



Tools and environment

- Windows 11;
- Docker Desktop with desktop-linux context.
- EMQX Enterprise 6.3.1 for MQTT messaging;
- broker dashboard at localhost:18083.
- TimescaleDB 2.30.2 on PostgreSQL 16;
- database smart_energy on host port 5432.
- Python 3.13.9 environments;
- psycopg 3.3.6 in the project .venv.
- Grafana at localhost:3000;
- PostgreSQL data source successfully tested.

Windows policy blocked psql.exe and a scikit-learn compiled DLL in .venv; use permitted Python runtimes.

2.2 MQTT and EMQX Ingestion Pilot

Why MQTT and what QoS 1 means:

A publisher sends a message to a topic; a broker receives and routes it to subscribers.

A subscriber listens to a topic filter such as energy/meters number.

QoS 1 means "at least once": delivery is acknowledged, but duplicates are possible.

The database therefore needs an explicit duplicate-handling strategy.

The pilot used JSON messages, QoS 1, and no retained messages.

MQTT pilot design:

Start the subscriber first so it is listening before the publisher sends records.

Publish the first 2,000 generated reading dictionaries.

Use topic energy/meters/{meter_id}; subscribe to energy/meters #.

Insert all eight reading fields into energy_live, including any duplicate delivery.

Verify publisher acknowledgements, subscriber receive count, and database row count.

Pilot and full-load route:

- Pilot target 2,000 Records for the end-to-end MQTT check
- Historical load 10,696,848 Raw rows inserted by direct DB writer, Raw meter coverage 4,000

Question 3. Database Design and Data Quality

3.1 Database Design and TimescaleDB Hypertables

Database design:

- meters stores meter metadata: meter_id, region, customer_type.
- energy_live is a normal PostgreSQL table for the pilot/operational path.
- energy_readings is the historical hypertable partitioned by event_time.
- The reading table has exactly eight fields: meter_id, event_time, received_time, power_kw, voltage_v, current_a, frequency_hz, energy_kwh.

The original generator was left unchanged.

Hypertables and chunks:

- A hypertable is a TimescaleDB table partitioned into time-based chunks.
- A chunk stores a bounded time range, allowing time filters to avoid unrelated data.
- Baseline interval: 1 day; comparison intervals: 3 hours and 1 week.
- Smaller chunks can improve time-local queries but create more chunk metadata.
- Larger chunks reduce chunk count but can increase work for narrow time windows.

3.2 Data Quality, Deduplication and Reconciliation

Raw data quality reconciliation

- Emitted records: 10,696,848

Actual raw rows loaded

- Unique meter-time readings: 10,643,735

Cleaned records after deduplication

- Duplicate extras: 53,113

Extra copies removed

Missing readings 108,265

Expected intervals absent; retained as gaps

- Duplicates, missing data, and delay:

- A duplicate is an additional record for an already-present (meter_id, event_time) key.
- A missing reading is an expected meter-time interval without a record.
- A delayed unique reading is a distinct reading arriving 30–180 minutes after event_time.
- Delayed unique readings: 213,090; these are a timeliness issue, not duplicates.

All eight fields were non-null in the loaded raw dataset, but expected intervals were still missing.

Deduplication method and integrity:

- Created energy_readings_raw_backup before deleting duplicate extras.
- Kept one row per (meter_id, event_time) and deleted extra physical copies.
- Created unique index uq_energy_readings_meter_event on (meter_id, event_time).
- Cleaned row count: 10,643,735; remaining duplicate groups: zero.

The raw-to-cleaned energy reconciliation differed only by floating-point rounding: about 0.000000033 kWh.

Cleaned dataset verification:

Clean rows: 10,643,735 Unique meter-time keys Meters represented 4,000 Distinct meter IDs

First event 1 Sep 2026 00:00 CAT / Africa-Kigali

Last event 28 Sep 2026 23:45

Final 15-minute interval

Why missing readings were not filled with zero

Zero means a real measurement of no energy use; missing means no observation exists.

Replacing missing with zero would artificially reduce totals and distort profiles.

Completeness is evaluated separately from energy consumption.

Question 4. Chunk-Interval Benchmarking

4.1 Benchmark Methodology and Query Definitions

Benchmark method: Q1–Q5

Q1: average power for each CAT hour on 21 September.

Q2: top 10 15-minute network demand intervals from 22–28 September, including ties crossing rank 10.

Q3: total energy per meter for the full period.

Q4: energy, average power, and peak power for meter 1000000000 during 22–28 September.

Q5: full-table row count, total energy, and average/minimum/maximum power.

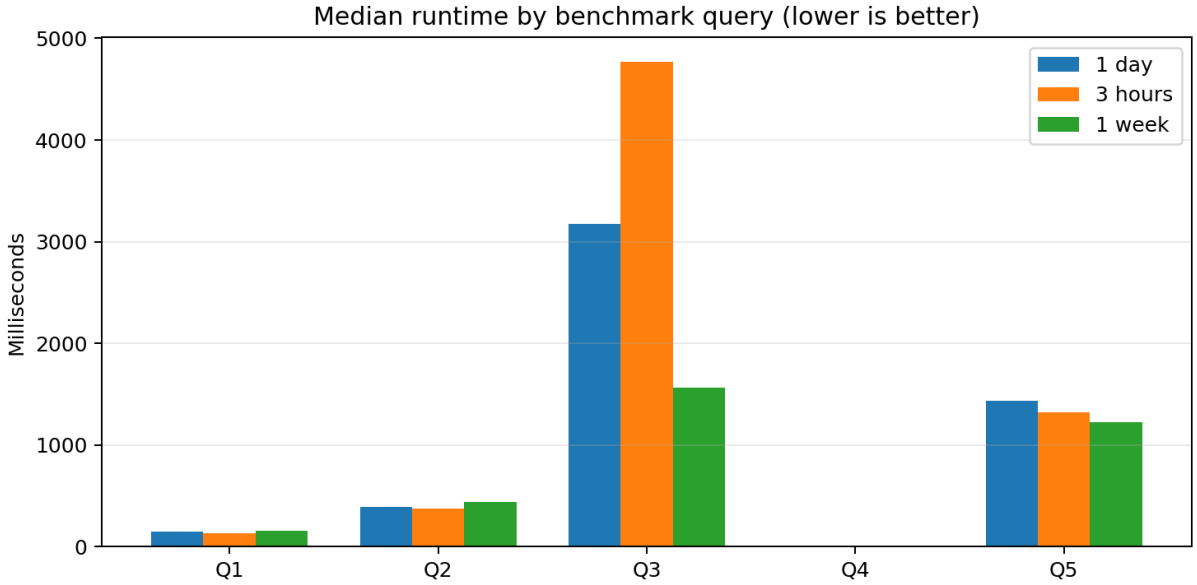
Five measured runs after warm-up; compare medians under consistent conditions.

4.2 Chunk-Interval Benchmark Results

The following table records the median query times measured for the three tested chunk intervals. Lower runtime is better. Percent improvements are relative to the named alternative and use the reported medians.

Query	1-day (ms)	3-hour (ms)	1-week (ms)	Winner
Q1: hourly average power	150.024	128.989	155.786	3-hour
Q2: top 15-minute network demand	387.484	377.933	441.449	3-hour
Q3: total energy per meter	3176.532	4771.378	1567.388	1-week
Q4: one meter, one-week window	3.952	11.586	4.056	1-day
Q5: full-dataset summary	1431.568	1319.707	1226.295	1-week

Query	Winner vs 1-day	Winner vs 3-hour / 1-week	Interpretation
Q1	14.02% faster than 1-day	17.20% faster than 1-week	3-hour suits time-windowed hourly aggregation.
Q2	2.46% faster than 1-day	14.39% faster than 1-week	3-hour gave the fastest demand ranking.
Q3	50.65% faster than 1-day	67.15% faster than 3-hour	1-week reduced overhead for broad meter totals.
Q4	—	65.88% faster than 3-hour; 2.56% faster than 1-week	1-day was fastest for the narrow single-meter window.
Q5	14.34% faster than 1-day	7.08% faster than 3-hour	1-week was fastest for the full-table summary.



Chunk benchmark: median runtime comparison

Q1 and Q2 are time-focused queries; the 3-hour layout was fastest in both.

What the chunk results mean

Q1: 3-hour chunks were about 14.02% faster than 1-day and 17.20% faster than 1-week.

Q2: 3-hour chunks were about 2.46% faster than 1-day and 14.39% faster than 1-week.

Q3: 1-week chunks were about 50.65% faster than 1-day and 67.15% faster than 3-hour.

Q4: 1-day chunks were about 65.88% faster than 3-hour and 2.56% faster than 1-week.

Q5: 1-week chunks were about 14.34% faster than 1-day and 7.08% faster than 3-hour.

There is no universal winner: access pattern determines which layout helps.

4.3 Chunk Storage and Operational Trade-offs

Chunk storage and trade-offs

- 1-day hypertable: 1,749 MB.
- 3-hour hypertable: 1,364 MB.
- 1-week hypertable: 1,350 MB.

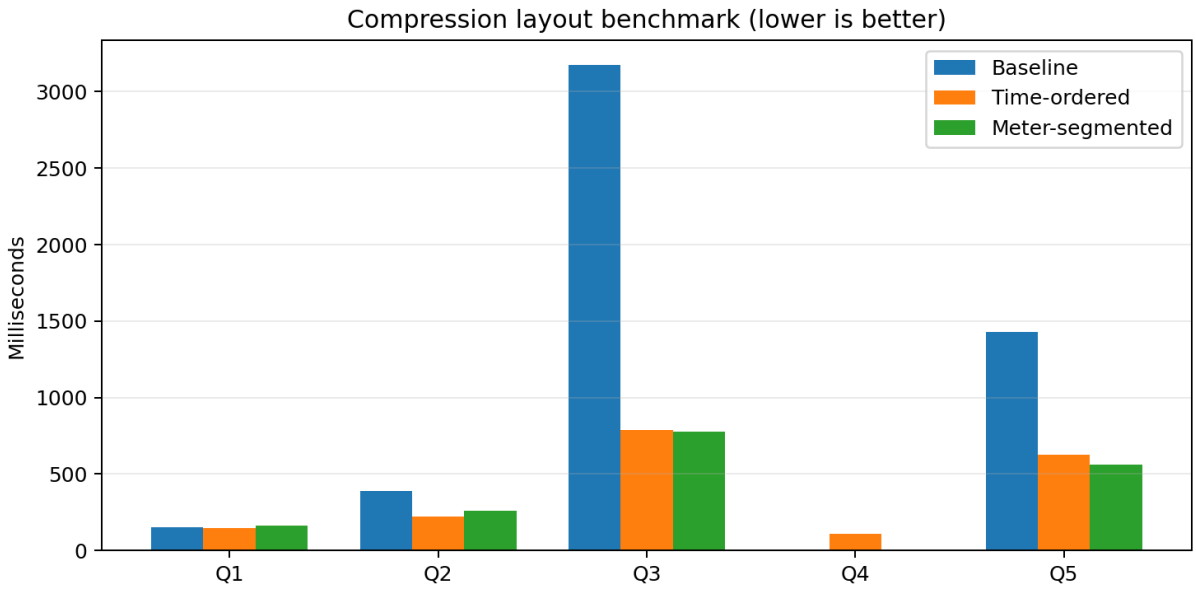
The 3-hour and 1-week copies were verified to match the cleaned baseline in row count, meters, time range, and totals.

Operational choice: retain the 1-day baseline for balanced simplicity, while recognizing workload-specific alternatives.

Question 5. Compression Design and Performance

5.1 Compression Design and Storage Results

Layout	Storage size	Reduction vs baseline	Relative size
Uncompressed baseline	1,749 MB	—	1.00×
Time-ordered compressed	524 MB	~70.0%	3.34× smaller
Meter-segmented compressed	590 MB	~66.3%	2.96× smaller



Compression design:

- Compression reduces storage by encoding repeated and ordered time-series values.
- energy_time_compressed: order by event_time DESC.
- energy_meter_compressed: order by event_time DESC and segment by meter_id.
- Both copies contain the same cleaned 10,643,735 readings.
- The same 1-day chunk interval was used to compare compression layout fairly.
- Only chunks whose end was more than 48 hours' old were eligible.

Compression storage results:

Baseline: 1,749 MB, Uncompressed cleaned hypertable

Time-ordered:

524 MB, ~70.0% reduction; 3.34× smaller

Meter-segmented: 590 MB ~66.3% reduction; 2.96× smaller

Compressed chunks 26 of 29, Three recent chunks remained uncompressed

5.2 Compression Query Performance

Compression: Q1–Q5 median runtime

How to interpret compression performance?

Time-oriented Q1 and Q2 favored time-ordered compression; Q2 fell from 387.484 to 220.712 ms.

Meter-oriented Q3 and Q4 favored meter segmentation; Q3 reached 778.665 ms.

Q5 full scan was fastest in the meter-segmented copy at 563.842 ms.

Compression layout is a workload choice, not merely a storage setting.

Compression improves storage substantially, but not every query benefits equally.

Question 6. Energy Analytics, Aggregates and Dashboards

6.1 Regional and Customer-Type Energy Analytics

Finding	Reported value / interpretation
Grouping	30 combinations: 10 regions × 3 customer types
Highest industrial regional total	Region 8, approximately 338,531.83 kWh
Commercial energy per registered meter	Region 8, approximately 862.49 kWh
Interpretation	Industrial customers use substantially more energy per registered meter; regional totals should be interpreted alongside meter counts and customer mix.

Energy analytics: regional and customer patterns

Joined cleaned readings to meters and grouped by region and customer_type.

Produced 30 combinations: 10 regions × 3 customer types.

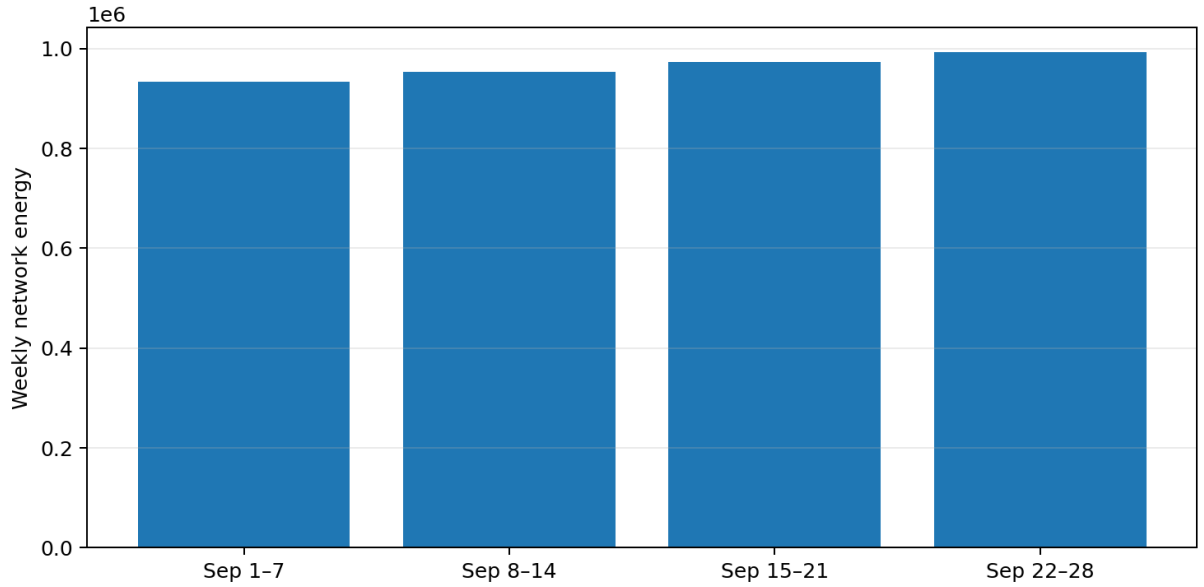
Calculated registered meter count, total energy, and energy per registered meter.

Industrial meters showed much higher per-meter energy than other customer groups.

Region 8 had the highest industrial total (~338,531.83 kWh) and highest commercial per-meter energy (~862.49 kWh).

6.2 Weekly Energy Trend and Network Demand Peaks

Week (CAT)	Network energy (kWh)
1–7 September 2026	934,199.060635
8–14 September 2026	953,714.972010
15–21 September 2026	973,230.026421
22–28 September 2026	992,560.287750



Rank	15-minute interval (CAT)	Network demand (kW)
1	28 Sep 2026, 13:00	7,993.691474
2	28 Sep 2026, 12:45	7,943.946545

3	28 Sep 2026, 13:15	7,943.635736

Network demand peaks

Highest 15-minute interval: 28 September 2026, 13:00 CAT — 7,993.691 kW summed across meters.

Second: 28 September, 12:45 CAT — 7,943.947 kW.

Third: 28 September, 13:15 CAT — 7,943.636 kW.

The three top intervals cluster around early afternoon, supporting peak-period operational planning.

These are measured network totals, not a forecast.

6.3 Regional Data Completeness

Completeness by region

Completeness compares observed readings with expected meter-time intervals.

Regional completeness was approximately 98.97%–99.01%.

Each region contained 400 registered meters.

A high percentage does not mean every meter is complete; gaps can cluster by meter or time.

Operational dashboards should show both energy and completeness.

6.4 Continuous Aggregates and Refresh Policies

Aggregate view	Bucket / grain	Rows	Refresh time
energy_15min_region	15-minute regional	26,880	57.229 s
energy_hour_region	Hourly regional	6,720	20.360 s
energy_day_meter	Daily per meter	112,000	26.232 s

Continuous aggregates: purpose

A continuous aggregate stores precomputed summaries over time buckets.

It avoids repeatedly scanning millions of raw readings for common reporting views.

Three views: 15-minute by region, hourly by region, and daily by meter.

Each stores total energy, total power, and reading count.

The production aggregates were initially created WITH NO DATA and refreshed manually for validation.

Continuous aggregate policies and reconciliation:

Continuous aggregate policies and reconciliation:

- Refresh policy lookback: 3 days.
- End offset: 1 hour; schedule interval: 15 minutes.
- Policies created for all three aggregate views.
- Aggregate totals reconciled against the cleaned source with zero count differences and only tiny floating-point differences.
- Production views remain materialized only = TRUE.

6.5 Materialized versus Real-Time Aggregate Freshness

Freshness experiment: materialized vs real-time

A separate test hypertable and hourly aggregate were used to isolate freshness behavior.

After refresh, an older hour contained 1 kWh / 1 row; an unmaterialized new hour was absent.

With real-time aggregation enabled, the new hour showed 2 kWh / 1 row while the older hour remained materialized.

After adding 0.5 kWh to the old hour, it stayed at 1 kWh until refresh; refresh updated it to 1.5 kWh / 2 rows.

An absent bucket means “not materialized / no returned row,” not automatically zero energy.

6.6 Grafana Dashboard Validation

Panel comparison	Maximum difference	Result
Network energy over time	6.46×10^{-11} kWh	PASS
Energy by region	5.53×10^{-10} kWh	PASS
Meter daily energy	0 kWh	PASS
15-minute demand	3.64×10^{-11} kW	PASS
Regional completeness	0 count; 0 percentage points	PASS

Grafana dashboard validation

Grafana was available at http://localhost:3000 with a PostgreSQL data source that passed Save & Test.

Raw panels query cleaned energy_readings; aggregate panels query the continuous aggregates.

Five equivalent panels covered network energy over time, regional energy, one meter’s daily energy, 15-minute demand, and regional completeness.

The selected reporting window was 22–29 September 2026 CAT, with all regions and meter 1000000000 where applicable.

Local Grafana OSS did not expose the Cloud/Enterprise query-result caching controls shown in the interface.

Question 7. Anomaly Detection

7.1 Anomaly Detection Method

Anomaly detection rule

For each meter and CAT 15-minute slot, estimate a baseline from the median on other days.

Use leave-one-day-out logic so the evaluated day does not define its own baseline.

Demand spike: observed power > 1.50 × baseline.

Unusual drop: observed power < 0.40 × baseline.

The detector is rule-based and explainable; the threshold is an explicit design choice.

7.2 Anomaly Detection Results and Interpretation

Measure	Result
Injected events	40 total: 20 demand spikes and 20 unusual drops
Event intervals detected	40 / 40 (100%)
Missed injected events	0
False-flagged readings	0 observed against supplied ground truth
Readings inside detected event intervals	474

Readings flagged	468
Interpretation	Six individual readings within detected spike intervals were not separately flagged; event-level detection is distinct from per-reading flagging.

Anomaly detection results

Injected events: 40

- 20 demand spikes + 20 unusual drops

Events detected

- 40 / 40

100% event-level detection

False-flagged readings: 0

0.0000% observed false-alarm rate

Within-event readings 474:

- 468 flagged; 6 spike readings not individually flagged

Anomaly result: important distinction

- All 40 injected event intervals were detected.

That does not mean every reading inside each event interval was flagged.

Six individual readings within demand-spike intervals were not separately flagged; unusual-drop intervals had no such misses.

The observed zero false flags apply to this generated dataset and chosen thresholds.

In production, false positives and missed events must be measured against verified operational labels.

Question 8. Forecasting and Error Analysis

8.1 Forecasting Objective and Dataset

Forecasting: objective and evaluation design

- Forecast next-day hourly regional energy, using one row per region and CAT hour.
- The dataset has 6,720 rows: 672 hourly timestamps × 10 regions.
- Chronological split: train 1–14 Sep; validation 15–21 Sep; test 22–28 Sep.
- Forecast origin is 00:00 at the start of each target day; all 24 hours are forecast at once.
- Test data remains unseen until model selection is frozen.
- Forecast dataset and leakage prevention
- Dataset columns: target_time, region, observed_energy_kwh, reading_count.
- No duplicate region-hour rows and no nulls in these dataset fields.
- Every region-hour had fewer than 1,600 readings because some readings were missing.
- Lag features were checked to ensure their source timestamps preceded forecast origin.
- No within-forecast-day lag_1h was used because that value would not be known at 00:00.

8.2 Forecast Features and Leakage Controls

Forecast feature configurations

Why compare on common training rows?

A longer lag needs a longer history before a row can be used.

Config A retained 3,120 rows; B retained 2,880; C retained 1,680.

The common training window was 8–14 September: 1,680 region-hour rows.

Using common rows prevents the model comparison from being confounded by different training periods.

All three lag-source checks reported zero invalid source timestamps.

8.3 Previous-Day Baseline and Random Forest Model

Previous-day baseline

Baseline formula: prediction(t) = observed energy at the same region and hour on the previous day.

It is simple, transparent, and a meaningful benchmark for daily patterns.

Test predictions: 1,680 = 7 days × 24 hours × 10 regions.

Baseline MAE: 42.708281 kWh.

Baseline RMSE: 79.402867 kWh.

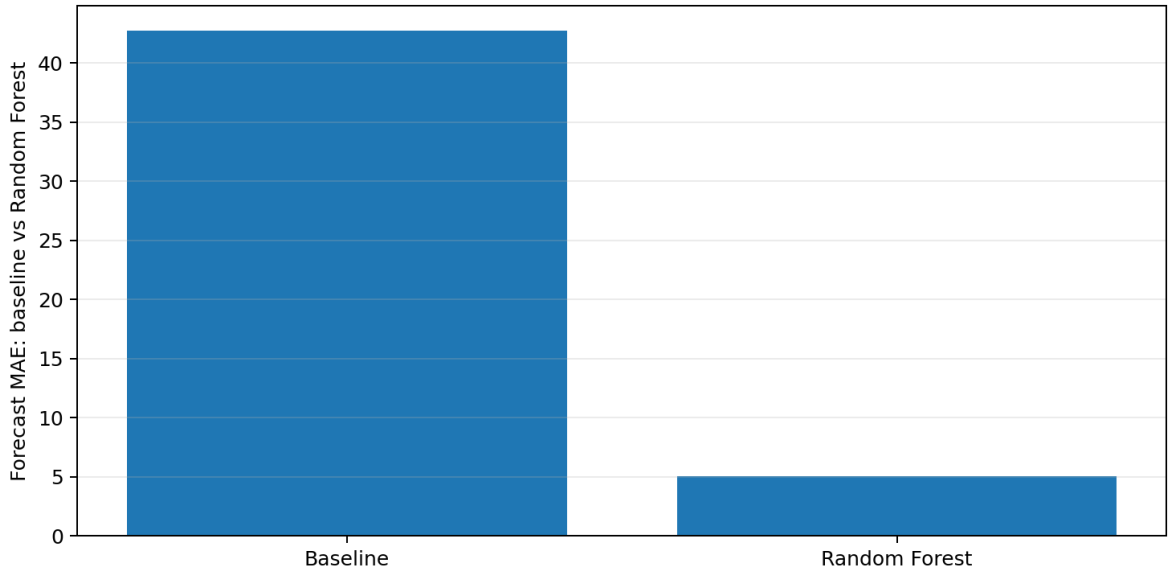
Forecast model and headline results Model Random Forest 200 trees; random state=42; n_jobs=-1

Baseline MAE :42.708281 kWh

Previous-day same-hour prediction

Random Forest MAE 5.054452 kWh Lower is better MAE improvement 88.17% and RMSE improved by 91.45%

Forecast error: baseline vs Random Forest The model reduced MAE by 37.653829 kWh and RMSE by 72.614879 kWh.



8.4 Regional Forecast Errors

Region	MAE (kWh)	RMSE (kWh)	Maximum absolute error (kWh)
8	8.215959	11.893355	49.996982

5	5.782	7.457	20.534
3	5.059	6.244	16.982
1	5.046	6.318	19.986
9	4.915	6.016	15.149
2	4.693	5.767	16.427
7	4.446	5.604	18.206
10	4.363	5.668	16.959
4	4.337	5.410	17.353
6	3.687761	4.748774	14.902

8.5 Forecast Interpretation and Error Analysis

Random Forest improved on the previous-day baseline in all 10 regions.

Region 8 was hardest: MAE 8.216 kWh, RMSE 11.893 kWh, maximum absolute error 49.997 kWh.

Region 6 was best: MAE 3.688 kWh and RMSE 4.749 kWh.

Mean signed errors were generally near zero; Region 8 slightly underpredicted overall by 1.320 kWh.

MAPE ranged approximately 0.73%–1.02%; one excellent hour does not establish overall performance.

Limitations: only seven test days, generated data, missing/delayed readings, and limited lag-only predictors.

8.6 Forecasting Recommendation

For this simulated workload, retain the previous-day method as a transparent benchmark and use the Random Forest model as the preferred forecasting candidate for further validation: its measured MAE was 5.054452 kWh versus 42.708281 kWh for the baseline, an 88.17% reduction, and its RMSE was 6.787988 kWh versus 79.402867 kWh, a 91.45% reduction. These results support further testing, not an unconditional production guarantee; validate with longer and real operational data before deployment.

Question 9. Recommendations, Limitations and Reproducibility

9.1 Operational Recommendations and Next Steps

Operational recommendation and next steps:

- Retain cleaned readings as the authoritative history; preserve gaps and monitor delayed arrival.
- Use compressed storage for older history and continuous aggregates for repeated dashboards.
- Keep the previous-day baseline as a transparent forecasting benchmark; monitor MAE and RMSE.
- Use forecast errors as investigation signals, not proof of anomalies.
- Add weather, holidays, industrial schedules, outages, and maintenance data before production deployment.
- The Random Forest is promising for this dataset, but further real-world and longer-horizon validation is required.

9.2 Limitations and Validity Considerations

Forecast interpretation and limitations:

Random Forest improved on the previous-day baseline in all 10 regions.

Region 8 was hardest: MAE 8.216 kWh, RMSE 11.893 kWh, maximum absolute error 49.997 kWh.

Region 6 was best: MAE 3.688 kWh and RMSE 4.749 kWh.

Mean signed errors were generally near zero; Region 8 slightly underpredicted overall by 1.320 kWh.

MAPE ranged approximately 0.73%–1.02%; one excellent hour does not establish overall performance.

Limitations: only seven test days, generated data, missing/delayed readings, and limited lag-only predictors.

Operational recommendation and next steps:

- Retain cleaned readings as the authoritative history; preserve gaps and monitor delayed arrival.
- Use compressed storage for older history and continuous aggregates for repeated dashboards.
- Keep the previous-day baseline as a transparent forecasting benchmark; monitor MAE and RMSE.
- Use forecast errors as investigation signals, not proof of anomalies.
- Add weather, holidays, industrial schedules, outages, and maintenance data before production deployment.
- The Random Forest is promising for this dataset, but further real-world and longer-horizon validation is required.

- The dataset is synthetic and the forecast test covers only 22–28 September 2026; results should not be treated as production guarantees.
- The source contains 108,265 missing expected readings and 213,090 delayed unique readings. Missing intervals remain gaps rather than being silently imputed as zero.
- The anomaly detector was assessed against 40 injected events; zero observed false-positive readings is a result for this test ground truth, not a guarantee of zero future false alarms.
- The Random Forest uses lag-based features and outperformed the baseline on this test, but additional seasons, operational conditions and external drivers should be evaluated before deployment.
- Chunk interval and compression choices are workload-dependent; the five queries do not establish one universal physical design.

9.3 Reproducibility and Evidence Checklist

- Confirm timezone is CAT / Africa-Kigali for all reporting windows.
- Record software versions and database configuration used for each run.
- Keep the raw backup and document duplicate-removal logic before cleaning.
- Verify uniqueness of (meter_id, event_time) after deduplication.
- Run each benchmark after warm-up and report all five runs plus median/minimum/maximum where available.
- Check compressed and aggregate tables against the cleaned source for row counts and numerical reconciliation.
- Preserve chronological train/validation/test separation and document lag-source timestamps.
- Retain Grafana screenshots, SQL query text, model configuration, and result outputs as evidence.

Tools and environment

Windows 11; Docker Desktop with desktop-linux context.

EMQX Enterprise 6.3.1 for MQTT messaging; broker dashboard at localhost:18083.

TimescaleDB 2.30.2 on PostgreSQL 16; database smart_energy on host port 5432.

Python 3.13.9 environments; psycopg 3.3.6 in the project .venv.

Grafana at localhost:3000; PostgreSQL data source successfully tested.

Windows policy blocked psql.exe and a scikit-learn compiled DLL in .venv; use permitted Python runtimes.

MQTT pilot design

Start the subscriber first so it is listening before the publisher sends records.

Publish the first 2,000 generated reading dictionaries.

Use topic energy/meters/{meter_id}; subscribe to energy/meters/#.

Insert all eight reading fields into energy_live, including any duplicate delivery.

Verify publisher acknowledgements, subscriber receive count, and database row count.

Deduplication method and integrity

Created energy_readings_raw_backup before deleting duplicate extras.

Kept one row per (meter_id, event_time) and deleted extra physical copies.

Created unique index uq_energy_readings_meter_event on (meter_id, event_time).

Cleaned row count: 10,643,735; remaining duplicate groups: zero.

The raw-to-cleaned energy reconciliation differed only by floating-point rounding: about 0.000000033 kWh.

Benchmark method: Q1–Q5

Q1: average power for each CAT hour on 21 September.

Q2: top 10 15-minute network demand intervals from 22–28 September, including ties crossing rank 10.

Q3: total energy per meter for the full period.

Q4: energy, average power, and peak power for meter 1000000000 during 22–28 September.

Q5: full-table row count, total energy, and average/minimum/maximum power.

Five measured runs after warm-up; compare medians under consistent conditions.

Continuous aggregate policies and reconciliation

Refresh policy lookback: 3 days.

End offset: 1 hour; schedule interval: 15 minutes.

Policies created for all three aggregate views.

Aggregate totals reconciled against the cleaned source with zero count differences and only tiny floating-point differences.

Production views remain materialized_only = TRUE.

Grafana dashboard validation

Grafana was available at http://localhost:3000 with a PostgreSQL data source that passed Save & Test.

Raw panels query cleaned energy_readings; aggregate panels query the continuous aggregates.

Five equivalent panels covered network energy over time, regional energy, one meter’s daily energy, 15-minute demand, and regional completeness.

The selected reporting window was 22–29 September 2026 CAT, with all regions and meter 1000000000 where applicable.

Local Grafana OSS did not expose the Cloud/Enterprise query-result caching controls shown in the interface.

Forecast dataset and leakage prevention

Dataset columns: target_time, region, observed_energy_kwh, reading_count.

No duplicate region-hour rows and no nulls in these dataset fields.

Every region-hour had fewer than 1,600 readings because some readings were missing.

Lag features were checked to ensure their source timestamps preceded forecast origin.

No within-forecast-day lag_1h was used because that value would not be known at 00:00.

9.4 Conclusion

What the chunk results mean

Q1: 3-hour chunks were about 14.02% faster than 1-day and 17.20% faster than 1-week.

Q2: 3-hour chunks were about 2.46% faster than 1-day and 14.39% faster than 1-week.

Q3: 1-week chunks were about 50.65% faster than 1-day and 67.15% faster than 3-hour.

Q4: 1-day chunks were about 65.88% faster than 3-hour and 2.56% faster than 1-week.

Q5: 1-week chunks were about 14.34% faster than 1-day and 7.08% faster than 3-hour.

There is no universal winner: access pattern determines which layout helps.

How to interpret compression performance

Time-oriented Q1 and Q2 favored time-ordered compression; Q2 fell from 387.484 to 220.712 ms.

Meter-oriented Q3 and Q4 favored meter segmentation; Q3 reached 778.665 ms.

Q5 full scan was fastest in the meter-segmented copy at 563.842 ms.

Compression layout is a workload choice, not merely a storage setting.

Compression improves storage substantially, but not every query benefits equally.

Completeness by region

Completeness compares observed readings with expected meter-time intervals.

Regional completeness was approximately 98.97%–99.01%.

Each region contained 400 registered meters.

A high percentage does not mean every meter is complete; gaps can cluster by meter or time.

Operational dashboards should show both energy and completeness.

Anomaly result: important distinction

All 40 injected event intervals were detected.

That does not mean every reading inside each event interval was flagged.

Six individual readings within demand-spike intervals were not separately flagged; unusual-drop intervals had no such misses.

The observed zero false flags applies to this generated dataset and chosen thresholds.

In production, false positives and missed events must be measured against verified operational labels.

Continuous aggregate policies and reconciliation

Refresh policy lookback: 3 days.

End offset: 1 hour; schedule interval: 15 minutes.

Policies created for all three aggregate views.

Aggregate totals reconciled against the cleaned source with zero count differences and only tiny floating-point differences.

Production views remain materialized_only = TRUE.

Raw vs aggregate dashboard checks:

Forecast interpretation and limitations

Random Forest improved on the previous-day baseline in all 10 regions.

Region 8 was hardest: MAE 8.216 kWh, RMSE 11.893 kWh, maximum absolute error 49.997 kWh.

Region 6 was best: MAE 3.688 kWh and RMSE 4.749 kWh.

Mean signed errors were generally near zero; Region 8 slightly underpredicted overall by 1.320 kWh.

MAPE ranged approximately 0.73%–1.02%; one excellent hour does not establish overall performance.

Limitations: only seven test days, generated data, missing/delayed readings, and limited lag-only predictors.

Operational recommendation and next steps

Retain cleaned readings as the authoritative history; preserve gaps and monitor delayed arrival.

Use compressed storage for older history and continuous aggregates for repeated dashboards.

Keep the previous-day baseline as a transparent forecasting benchmark; monitor MAE and RMSE.

Use forecast errors as investigation signals, not proof of anomalies.

Add weather, holidays, industrial schedules, outages, and maintenance data before production deployment.

The Random Forest is promising for this dataset, but further real-world and longer-horizon validation is required.

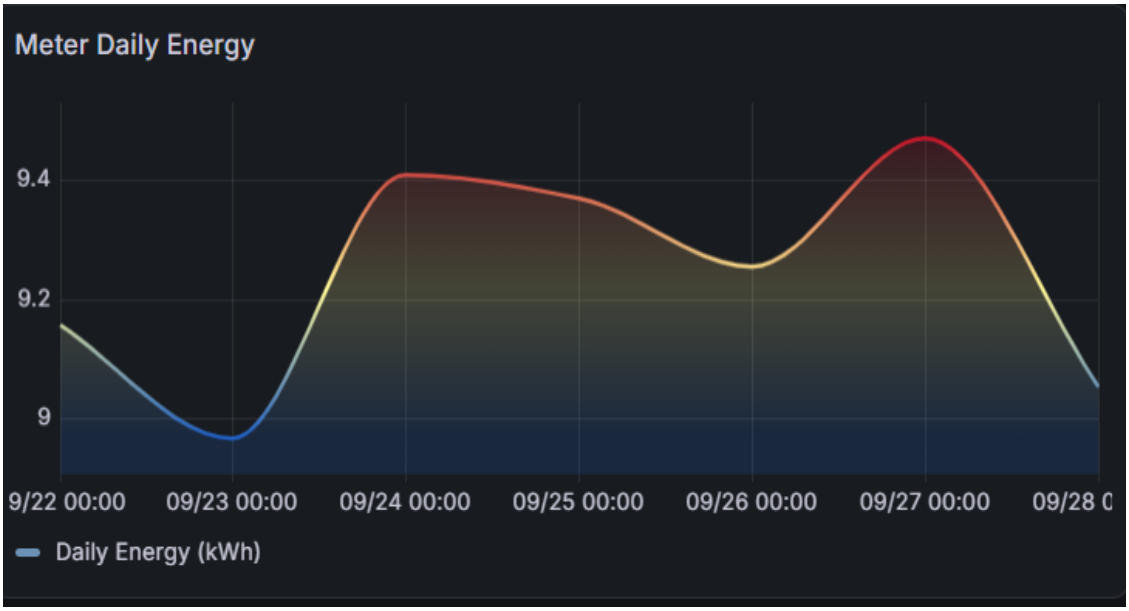
Overall, the measured work demonstrates an end-to-end smart-meter analytics workflow with explicit data-quality controls, query-specific physical design evaluation, storage compression, reusable aggregates, dashboard reconciliation, rule-based anomaly detection and a leakage-aware forecasting experiment. The most defensible operational choice is to keep the cleaned history authoritative, preserve missingness and arrival-delay information, use continuous aggregates for repeated summaries, and select chunk/compression layouts based on the actual query mix.

Additional Research Material and Appendices and Analysis Chartd

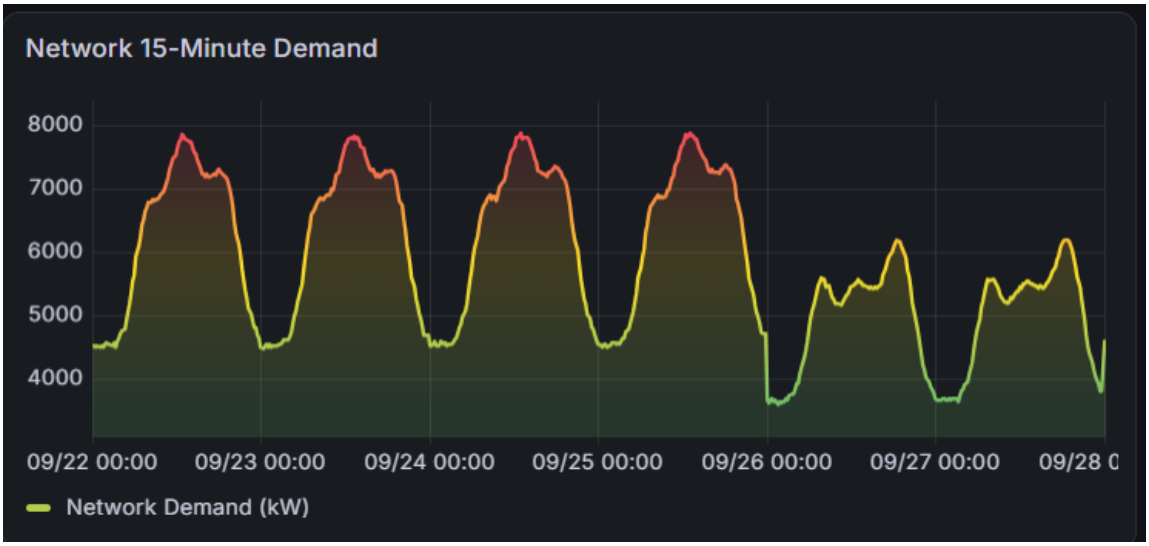
1. Completeness by Region



2. Meter Daily Energy



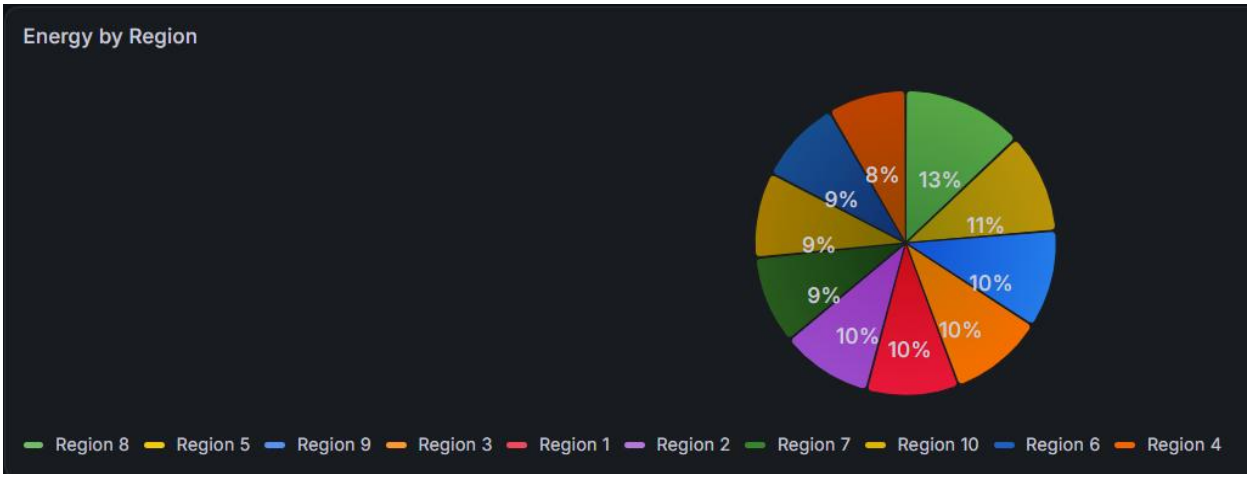
3. **Network 15 Minute Demand**



4 Network over Time



5. Energy Network by Region



Supplementary Note A — Advanced Model Comparisons

Compare Random Forest with Gradient Boosting, XGBoost, LightGBM, neural networks and temporal deep-learning models, where appropriate.

These comparisons should use the same target definition, time split and evaluation metrics.

Supplementary Note B — Distributed Big Data Architecture

The system could be extended to evaluate a distributed pipeline involving:

```
Kafka
Kafka Connect
Spark Structured Streaming
HDFS or object storage
```

This would enable a comparison between database-centered time-series analytics and a distributed streaming-and-storage architecture.

The supplied report does not claim that this extension was implemented in the current study.

Supplementary Conclusion

The SmartEnergyGridAnalytics project developed and evaluated an integrated smart-energy analytics workflow covering data generation, MQTT communication, time-series storage, data quality, chunk optimization, compression, analytical querying, continuous aggregates, dashboard validation, anomaly detection and machine-learning forecasting.

The final cleaned dataset contained 10,643,735 unique readings from 4,000 meters. The data-quality analysis identified 53,113 duplicate records, 108,265 missing readings and 213,090 delayed unique readings. Duplicate removal reconciled against the raw backup to a negligible reported floating-point difference. Missing readings were retained as gaps rather than interpreted as zero consumption.

The chunk-size experiment showed that the optimal physical design depends on the analytical workload. Three-hour chunks performed best for Q1 and Q2, one-week chunks performed best for Q3 and Q5, and one-day chunks performed best for Q4. One-day chunks were selected as the balanced operational baseline rather than as a universal winner.

Compression produced substantial storage savings. Time-ordered compression reduced the reported baseline of 1,749 MB to 524 MB, approximately 70.0%. Meter-segmented compression reduced storage to 590 MB, approximately 66.3%. The compressed-query results also showed that the fastest configuration depended on the query.

Continuous aggregates provided 15-minute regional, hourly regional and daily meter summaries. Their reported row counts matched expectations, and the five raw-versus-aggregate dashboard comparisons showed negligible numerical differences. The freshness experiment demonstrated that a missing materialized bucket does not necessarily mean zero consumption.

The anomaly detector identified all 40 injected anomaly events, with zero observed false-positive readings against the known ground truth. Six individual readings within detected demand-spike intervals were not separately flagged, illustrating why event-level and reading-level detection must be reported separately.

The forecasting experiment produced the strongest predictive result. Random Forest reduced MAE from 42.708281 kWh to 5.054452 kWh, an improvement of 88.17%. RMSE decreased from 79.402867 kWh to 6.787988 kWh, an improvement of 91.45%.

The regional error analysis identified Region 8 as the most difficult region, with MAE of 8.215959 kWh, RMSE of 11.893355 kWh and maximum error of 49.996982 kWh. Region 6 had the lowest reported MAE and RMSE. These results provide a basis for targeted investigation, but they do not establish the causes of the regional differences.

Consequently, Random Forest is recommended as a promising forecasting candidate for the simulated workload. MAE should be the primary operational metric, RMSE should be secondary, and the previous-day baseline should remain available for comparison.

The study does not establish universal forecasting accuracy or production readiness. The dataset is simulated, the historical period covers 28 days, the test period covers seven days, and missing and delayed readings remain important operational considerations.

The principal outcome is an integrated analytical architecture in which data quality protects correctness, time-series storage supports historical analysis, compression controls storage requirements, continuous aggregates support repeated reporting, anomaly detection identifies unusual behavior, forecasting supports planning, and dashboards make the results accessible.

Supplementary Reproducibility and Evidence Checklist

The following checklist organizes the evidence needed to reproduce and defend the study. It lists evidence to retain; it does not claim that every screenshot or source file is included in the supplied report.

a) Environment and ingestion

- Docker Desktop and container status.
- EMQX configuration and successful connection.
- TimescaleDB version and database connection.
- Africa/Kigali time-zone configuration.
- MQTT publisher and subscriber code.
- Topic configuration and QoS 1.
- Evidence of the 2,000-record pilot.
- Evidence distinguishing the pilot from direct historical loading.

b) Data generation and quality

- Simulator configuration.
- Planned count: 10,752,000.
- Emitted count: 10,696,848.
- Duplicate count: 53,113.
- Cleaned count: 10,643,735.
- Missing-reading analysis.
- Delayed-reading analysis.
- Raw backup and energy reconciliation.
- Four-thousand-meter validation.

c) Database design and benchmarking

- Table definitions and field types.
- Hypertable configuration.
- One-day, three-hour and one-week chunk configurations.
- Indexes and logical uniqueness rule.
- Benchmark SQL for Q1–Q5.
- Warm-up and five measured executions.
- Medians and the available minimum/maximum values.
- Percentage-improvement calculations.
- Storage measurements and compression settings.

d) Analytics and anomaly detection

- Regional and customer-type analysis.
- Daily and seven-day energy results.
- Top network-demand intervals.
- Regional completeness.
- Anomaly threshold configuration.
- Event-level and reading-level results.
- Evidence of the six individual readings not separately flagged.

e) Continuous aggregates and dashboards

- Definitions of all three continuous aggregates.

- Row counts and refresh times.
- Refresh-policy settings.
- Freshness experiment results.
- Grafana PostgreSQL data source.
- Raw-versus-aggregate panel queries.
- Five-panel numerical comparison.

f) Forecasting

- Dataset preparation and chronological split.
- Lag-feature generation.
- Forecast-origin timing checks.
- Previous-day baseline.
- Random Forest configuration.
- Validation-based model selection.
- Frozen test evaluation.
- MAE and RMSE comparison.
- Regional error and bias analysis.
- Forecasting limitations and recommendation.

h) Appendices: code examples and analytical result references

The following examples are based on the reported database structure and methods. They illustrate how to reproduce selected checks. They are not claimed to be verbatim copies of the original project source files.

Appendix A — Database connection and initial validation

The report used Python with psycopg to perform database operations.

```
import psycopg

# Replace connection details with the actual configured values.
# Do not place real passwords in shared source files.
conn = psycopg.connect(
    host="localhost",
    port=5432,
    dbname="smart_energy",
    user="Ildephonse",
    password="YOUR_CONFIGURED_PASSWORD"
)

try:
    with conn.cursor() as cur:
        # Verify the database server version.
        cur.execute("SELECT version();")
        print("Database version:", cur.fetchone()[0])

        # Verify the configured session time zone.
        cur.execute("SHOW TIME ZONE;")
        print("Time zone:", cur.fetchone()[0])
finally:
    conn.close()
```

Purpose: confirms that Python can connect to the database and checks the database version and session time zone.

The password placeholder must be replaced with the actual configured credential when executing the example.

Appendix B — Validate the cleaned dataset

```
-- Count cleaned readings.
SELECT COUNT(*) AS cleaned_rows
FROM energy_readings;

-- Count distinct meters.
SELECT COUNT(DISTINCT meter_id) AS distinct_meters
FROM energy_readings;

-- Check for duplicate logical readings.
SELECT COUNT(*) AS duplicate_groups
FROM (
    SELECT meter_id, event_time
    FROM energy_readings
    GROUP BY meter_id, event_time
    HAVING COUNT(*) > 1
) AS duplicate_readings;
```

The expected reported checks are:

- cleaned rows: 10,643,735;
- distinct meters: 4,000;
- duplicate groups after cleaning: zero.

These queries are verification examples based on the reported table and logical key.

Appendix C — Verify energy reconciliation

```
-- Compare energy totals in the raw backup and cleaned table.
-- The column name is based on the supplied report.

SELECT
    'raw_backup' AS dataset,
    SUM(energy_kwh) AS total_energy_kwh
FROM energy_readings_raw_backup

UNION ALL

SELECT
    'cleaned' AS dataset,
    SUM(energy_kwh) AS total_energy_kwh
FROM energy_readings;
```

The reported totals were approximately:

```
Raw backup: 3,872,863.893179 kWh
Cleaned:    3,853,704.346816 kWh
```

The difference represents energy associated with removed duplicate records, subject to the negligible floating-point reconciliation difference already described.

Appendix D — Calculate query-performance improvement

```
def improvement_percent(alternative_ms, winner_ms):
    """
    Calculate the percentage reduction in execution time
    relative to an alternative configuration.
    A positive result means the winner is faster.
    """
    return (
        (alternative_ms - winner_ms)
        / alternative_ms
        * 100
```

```
    )

# Q1: three-hour chunks versus one-day chunks.
q1_improvement = improvement_percent(
    alternative_ms=150.024,
    winner_ms=128.989
)
print(f"Q1 improvement: {q1_improvement:.2f}%")
```

Expected output:

```
Q1 improvement: 14.02%
```

The function uses measured values supplied in the report. It does not run a database benchmark.

Appendix E — Calculate forecasting metrics

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error


# y_true must contain the actual test observations.
# y_pred must contain predictions for the same observations.
# Both arrays must use the same ordering and units.
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))


print("MAE:", mae)
print("RMSE:", rmse)
```

The code illustrates how the reported metrics can be calculated from aligned arrays of actual and predicted observations. The original test arrays were not included in the supplied report, so this example does not independently reproduce the reported numerical results.

The reported results remain:

Model	MAE	RMSE
Previous-day baseline	42.708281 kWh	79.402867 kWh
Random Forest	5.054452 kWh	6.787988 kWh

Appendix F — Final results reference

Study results at a glance

- Cleaned readings: 10,643,735
- 4,000 meters
- Time compression: 70.0% — reported storage reduction
- Injected anomaly events: 40/40 — detected at event level
- Forecast MAE improvement: 88.17% — versus previous-day baseline

All values above come from the supplied report. The anomaly result is event-level; six individual readings within detected demand-spike intervals were not separately flagged.

Category	Final reported result
Planned readings	10,752,000
Emitted readings	10,696,848
Duplicate records removed	53,113
Missing readings	108,265
Delayed unique readings	213,090
Best chunk for Q1	3-hour

Best chunk for Q2	3-hour
Best chunk for Q3	1-week
Best chunk for Q4	1-day
Best chunk for Q5	1-week
Baseline storage	1,749 MB
Time-compressed storage	524 MB
Meter-segmented storage	590 MB
Injected events detected	40 of 40
False-positive readings reported	0
Baseline MAE	42.708281 kWh
Random Forest MAE	5.054452 kWh
Baseline RMSE	79.402867 kWh
Random Forest RMSE	6.787988 kWh
Hardest forecasting region	Region 8
Lowest-error forecasting region	Region 6

Final research statement

The SmartEnergyGridAnalytics study demonstrates the analytical lifecycle of a large IoT energy dataset, from data generation and ingestion to historical storage, quality control, performance optimization, monitoring and prediction.

The empirical findings support TimescaleDB for time-series organization, compression for substantial storage reduction, continuous aggregates for repeated reporting, independent anomaly detection for unusual behavior, and Random Forest as a promising short-term forecasting model under the tested conditions.

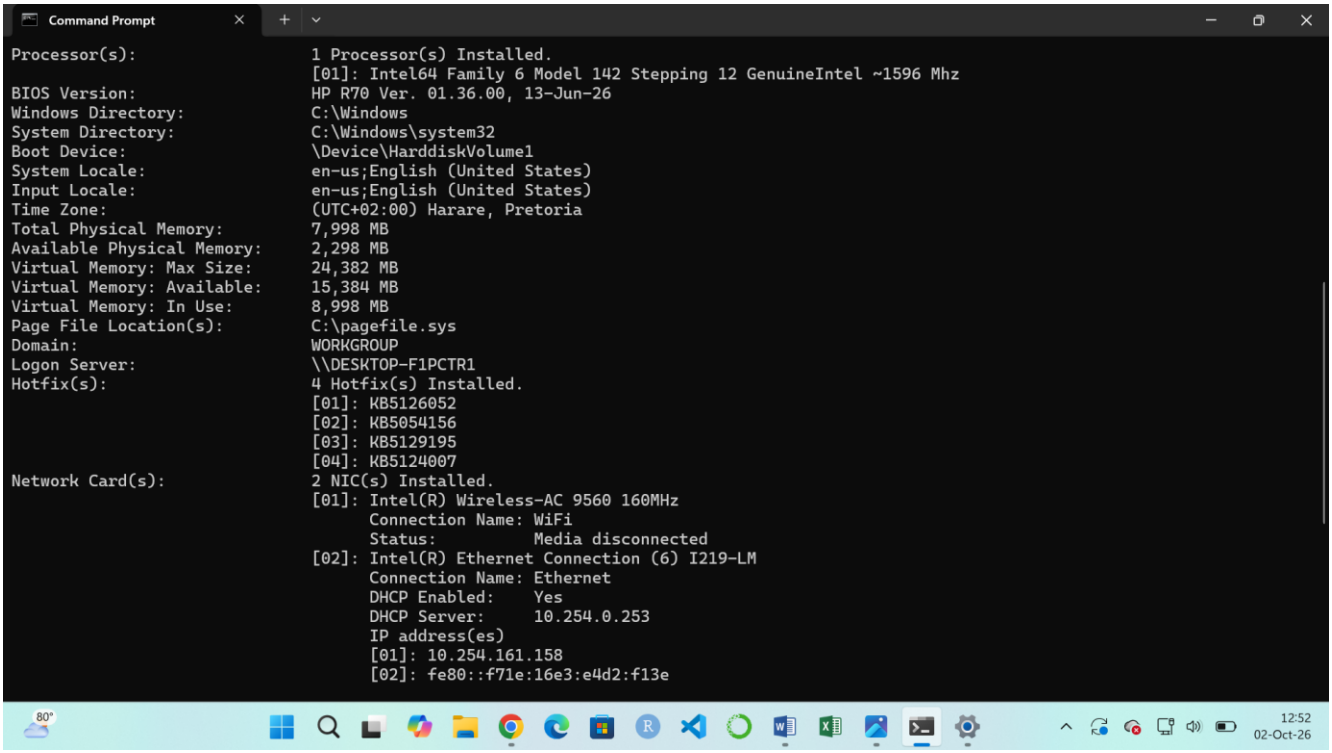
The conclusions must remain bounded by the evidence: the dataset is simulated, the observation period is limited, missing and delayed readings exist, and forecasting was evaluated on only seven test days.

The recommended system is therefore not simply a database or a machine-learning model. It is an integrated smart-energy analytics platform in which:

- Data quality protects analytical correctness.
- Time-series storage supports historical queries.
- Chunking and compression balance query performance and storage requirements.
- Continuous aggregates support efficient repeated reporting.
- Anomaly detection supports investigation of unusual behavior.
- Forecasting supports demand planning.
- Dashboards connect the analytical results to operational decisions.

This integrated architecture is the principal outcome of the SmartEnergyGridAnalytics project.

G. Screen Short:




```
C:\Windows\System32>pip install paho-mqtt==1.6.1
Collecting paho-mqtt==1.6.1
  Downloading paho-mqtt-1.6.1.tar.gz (99 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: paho-mqtt
  Building wheel for paho-mqtt (pyproject.toml) ... done
  Created wheel for paho-mqtt: filename=paho_mqtt-1.6.1-py3-none-any.whl size=65755 sha256=8fca5ea2c89880b77e1dbc63bdd5d7780db8fe339b1b5e77600737485923421f
  Stored in directory: c:\users\akioli technology\appdata\local\pip\cache\wheels\23\d5\af\1f3cbcc350dec9d8e95040f388e0163d132eff0c9a453db659
Successfully built paho-mqtt
Installing collected packages: paho-mqtt
Successfully installed paho-mqtt-1.6.1

[notice] A new release of pip is available: 26.1.1 -> 26.2.1
[notice] To update, run: C:\Users\AKIOLI TECHNOLOGY\AppData\Local\Programs\Python\Python312\python.exe -m pip install --upgrade pip

C:\Windows\System32>
```

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\AKIOLI TECHNOLOGY> D:

D:\>mkdir smart_energy_grid_analytics

D:\>cd smart_energy_grid_analytics

D:\smart_energy_grid_analytics>mkdir code sql data results screenshots charts report

D:\smart_energy_grid_analytics>python -m venv .venv

D:\smart_energy_grid_analytics>.venv/Scripts/activate
'.venv' is not recognized as an internal or external command,
operable program or batch file.

D:\smart_energy_grid_analytics>.venv\Scripts\activate

(.venv) D:\smart_energy_grid_analytics>python --version
Python 3.13.9

(.venv) D:\smart_energy_grid_analytics>where python
D:\smart_energy_grid_analytics\.venv\Scripts\python.exe
C:\Program Files\PyManager\python.exe
C:\Users\AKIOLI TECHNOLOGY\AppData\Local\Programs\Python\Python312\python.exe

(.venv) D:\smart_energy_grid_analytics>
```




```
C:\Windows\System32>pip install paho-mqtt==1.6.1
Collecting paho-mqtt==1.6.1
  Downloading paho-mqtt-1.6.1.tar.gz (99 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: paho-mqtt
  Building wheel for paho-mqtt (pyproject.toml) ... done
  Created wheel for paho-mqtt: filename=paho_mqtt-1.6.1-py3-none-any.whl size=65755 sha256=8fca5ea2c89880b77e1dbc63bdd5d7780db8fe339b1b5e77600737485923421f
  Stored in directory: c:\users\akioli technology\appdata\local\pip\cache\wheels\23\d5\af\1f3cbcc350dec9d8e95040f388e0163d132eff0c9a453db659
Successfully built paho-mqtt
Installing collected packages: paho-mqtt
Successfully installed paho-mqtt-1.6.1

[notice] A new release of pip is available: 26.1.1 -> 26.2.1
[notice] To update, run: C:\Users\AKIOLI TECHNOLOGY\AppData\Local\Programs\Python\Python312\python.exe -m pip install --upgrade pip

C:\Windows\System32>
```



```
(.venv) D:\smart_energy_grid_analytics>python -m pip install --upgrade pip
Requirement already satisfied: pip in d:\smart_energy_grid_analytics\.venv\lib\site-packages (25.2)
Collecting pip
  Using cached pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
Using cached pip-26.2.1-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2
Successfully installed pip-26.2.1

(.venv) D:\smart_energy_grid_analytics>python -m pip install paho-mqtt pyscopg2-binary
Collecting paho-mqtt
  Using cached paho_mqtt-2.1.0-py3-none-any.whl.metadata (23 kB)
ERROR: Could not find a version that satisfies the requirement pyscopg2-binary (from versions: none)
ERROR: No matching distribution found for pyscopg2-binary

(.venv) D:\smart_energy_grid_analytics>python -m pip install paho-mqtt
Collecting paho-mqtt
  Using cached paho_mqtt-2.1.0-py3-none-any.whl.metadata (23 kB)
Using cached paho_mqtt-2.1.0-py3-none-any.whl (67 kB)
Installing collected packages: paho-mqtt
Successfully installed paho-mqtt-2.1.0

(.venv) D:\smart_energy_grid_analytics>
```




```
(.venv) D:\smart_energy_grid_analytics>python -m pip install psycopg2-binary
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.13-cp313-cp313-win_amd64.whl.metadata (5.1 kB)
Downloading psycopg2_binary-2.9.13-cp313-cp313-win_amd64.whl (2.8 MB)
   ---------------------------------------- 2.8/2.8 MB 636.9 kB/s  0:00:05
Installing collected packages: psycopg2-binary
Successfully installed psycopg2-binary-2.9.13

(.venv) D:\smart_energy_grid_analytics>
```




```
                                    NAMES
eee08a8f11f0   emqx/emqx-enterprise:6.3.1   "/usr/bin/docker-ent…"   4 days ago   Up 40 minutes   0.0.0.0:1883->1883/tcp, [::]:1883->
1883/tcp, 0.0.0.0:8083-8084->8083-8084/tcp, [::]:8083-8084->8083-8084/tcp, 0.0.0.0:8883->8883/tcp, [::]:8883->8883/tcp, 0.0.0.0:18083
->18083/tcp, [::]:18083->18083/tcp   emqx

C:\Users\AKIOLI TECHNOLOGY>docker pull timescale/timescaledb:latest-pg16
latest-pg16: Pulling from timescale/timescaledb
Digest: sha256:6f139d56042989bd35f50ba5986e492cd32e35cc6778c50be2659add298dc09f
Status: Image is up to date for timescale/timescaledb:latest-pg16
docker.io/timescale/timescaledb:latest-pg16

C:\Users\AKIOLI TECHNOLOGY>
```




```
                                    NAMES
eee08a8f11f0   emqx/emqx-enterprise:6.3.1   "/usr/bin/docker-ent…"   4 days ago   Up 40 minutes   0.0.0.0:1883->1883/tcp, [::]:1883->
1883/tcp, 0.0.0.0:8083-8084->8083-8084/tcp, [::]:8083-8084->8083-8084/tcp, 0.0.0.0:8883->8883/tcp, [::]:8883->8883/tcp, 0.0.0.0:18083
->18083/tcp, [::]:18083->18083/tcp   emqx

C:\Users\AKIOLI TECHNOLOGY>docker pull timescale/timescaledb:latest-pg16
latest-pg16: Pulling from timescale/timescaledb
Digest: sha256:6f139d56042989bd35f50ba5986e492cd32e35cc6778c50be2659add298dc09f
Status: Image is up to date for timescale/timescaledb:latest-pg16
docker.io/timescale/timescaledb:latest-pg16

C:\Users\AKIOLI TECHNOLOGY>
```




```
======================================================================
FROZEN MODEL TEST RESULTS
======================================================================
Frozen configuration: B
Features: lag_24h, lag_48h, lag_168h
Training rows: 1,680
Test predictions: 1,680
Test MAE:  5.054452
Test RMSE: 6.787988

======================================================================
SAMPLE TEST PREDICTIONS
======================================================================
             target_time   region  observed_energy_kwh  prediction     error  absolute_error
2026-09-22 00:00:00+02:00 Region 1           435.589852  443.067521  7.477669        7.477669
2026-09-22 01:00:00+02:00 Region 1           438.892476  440.814887  1.922411        1.922411
2026-09-22 02:00:00+02:00 Region 1           437.375285  444.068849  6.693564        6.693564
2026-09-22 03:00:00+02:00 Region 1           447.844121  443.257873 -4.586248        4.586248
2026-09-22 04:00:00+02:00 Region 1           463.196811  466.301500  3.104689        3.104689
2026-09-22 05:00:00+02:00 Region 1           509.526680  514.081203  4.554523        4.554523
2026-09-22 06:00:00+02:00 Region 1           586.374958  589.589202  3.214245        3.214245
2026-09-22 07:00:00+02:00 Region 1           645.063962  650.595244  5.531282        5.531282
2026-09-22 08:00:00+02:00 Region 1           672.884711  678.839900  5.955189        5.955189
2026-09-22 09:00:00+02:00 Region 1           698.812723  689.335257 -9.477466        9.477466

======================================================================
FILES SAVED
======================================================================
D:\smart_energy_grid_analytics\code\forecasting\frozen_regression_test_predictions.csv
D:\smart_energy_grid_analytics\code\forecasting\frozen_regression_test_results.csv

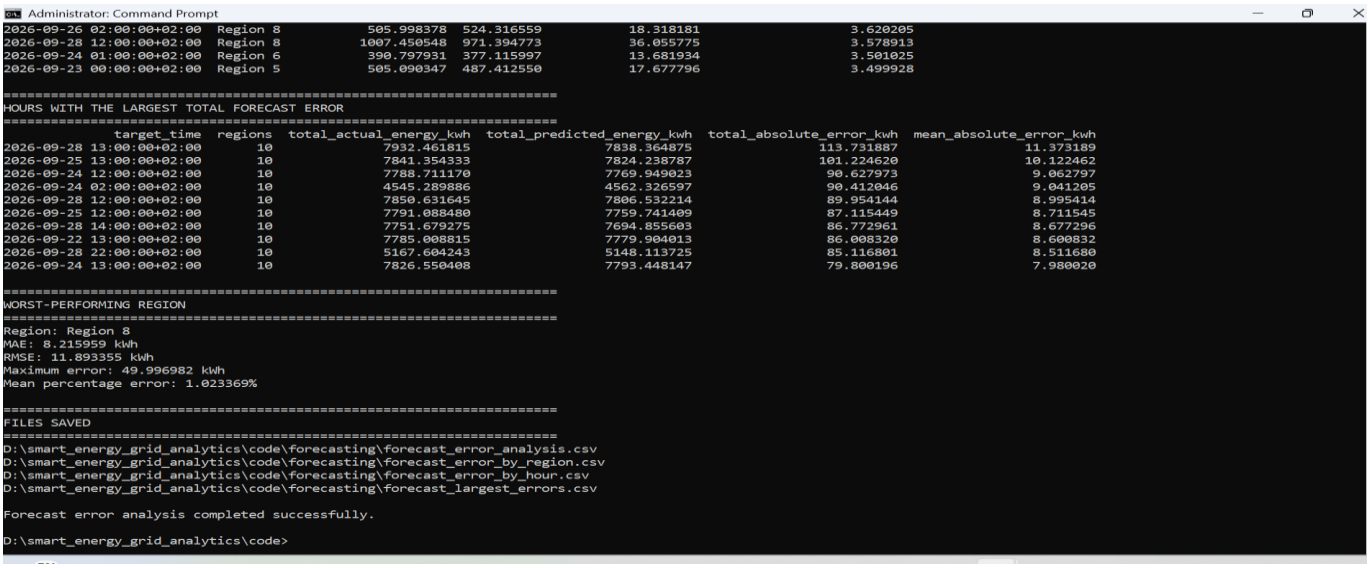
Done.
```



```
Administrator: Command Prompt - psql  -h localhost -p 5432 -U lldephonse -d smart_energy
smart_energy-#     COUNT(*) AS row_count,
smart_energy-#     COUNT(DISTINCT meter_id) AS distinct_meters,
smart_energy-#     MIN(event_time) AS earliest_event,
smart_energy-#     MAX(event_time) AS latest_event,
smart_energy-#     SUM(power_kw) AS total_power,
smart_energy-#     SUM(energy_kwh) AS total_energy
smart_energy-# FROM energy_readings
smart_energy-#
smart_energy-# UNION ALL
smart_energy-#
smart_energy-# SELECT
smart_energy-#     'energy_time_compressed',
smart_energy-#     COUNT(*),
smart_energy-#     COUNT(DISTINCT meter_id),
smart_energy-#     MIN(event_time),
smart_energy-#     MAX(event_time),
smart_energy-#     SUM(power_kw),
smart_energy-#     SUM(energy_kwh)
smart_energy-# FROM energy_time_compressed
smart_energy-#
smart_energy-# UNION ALL
smart_energy-#
smart_energy-# SELECT
smart_energy-#     'energy_meter_compressed',
smart_energy-#     COUNT(*),
smart_energy-#     COUNT(DISTINCT meter_id),
smart_energy-#     MIN(event_time),
smart_energy-#     MAX(event_time),
smart_energy-#     SUM(power_kw),
smart_energy-#     SUM(energy_kwh)
smart_energy-# FROM energy_meter_compressed;
      table_name        | row_count | distinct_meters |     earliest_event     |     latest_event       |    total_power     |    total_energy
------------------------+-----------+-----------------+------------------------+------------------------+--------------------+--------------------
 energy_meter_compressed |  10643735 |            4000 | 2026-09-01 00:00:00+02 | 2026-09-28 23:45:00+02 |    15414817.387793 |  3853704.346815414
 energy_time_compressed  |  10643735 |            4000 | 2026-09-01 00:00:00+02 | 2026-09-28 23:45:00+02 | 15414817.387793003 |  3853704.346815413
 energy_readings         |  10643735 |            4000 | 2026-09-01 00:00:00+02 | 2026-09-28 23:45:00+02 | 15414817.387793005 | 3853704.3468154133
(3 rows)


smart_energy=#
```





```
Administrator: Command Prompt
2026-09-26 02:00:00+02:00  Region 8           505.998378  524.316559          18.318181                3.620205
2026-09-28 12:00:00+02:00  Region 8          1007.450548  971.394773          36.055775                3.578913
2026-09-24 01:00:00+02:00  Region 6           390.797931  377.115997          13.681934                3.501025
2026-09-23 00:00:00+02:00  Region 5           505.090347  487.412550          17.677796                3.499928

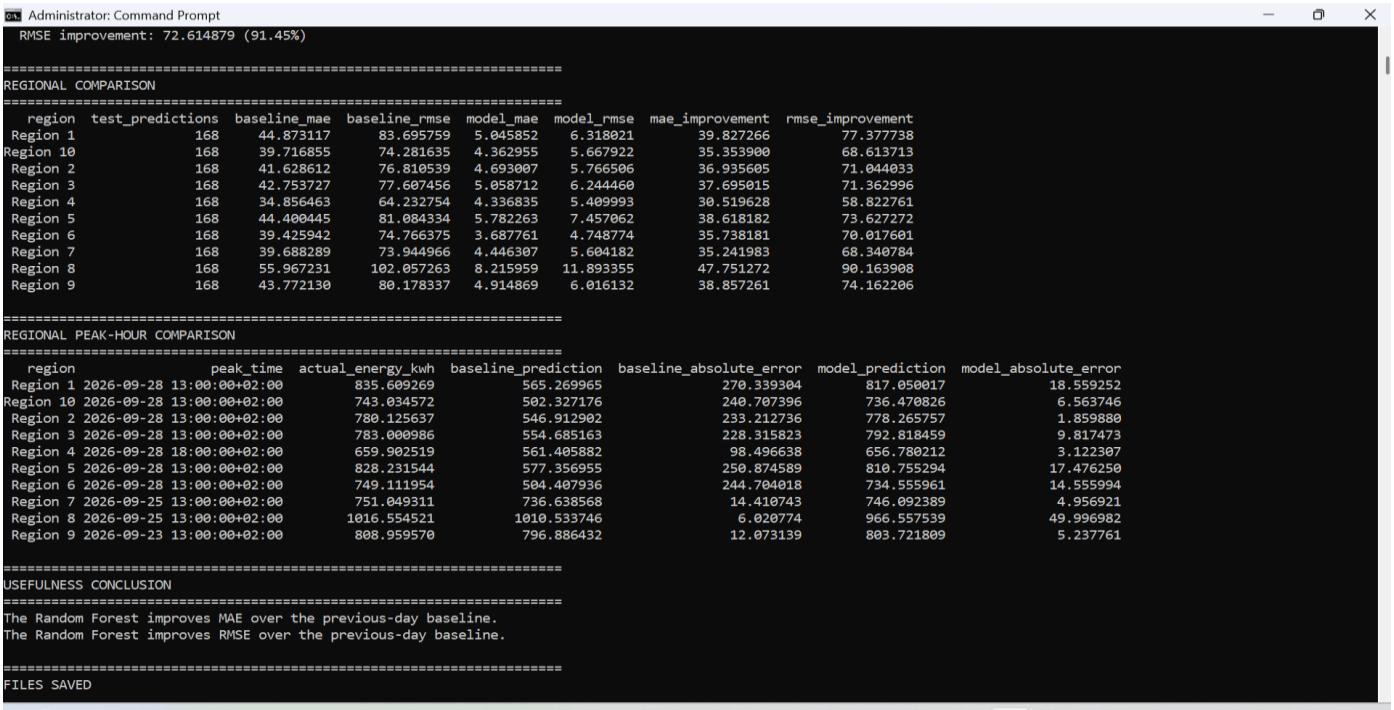
================================================================
HOURS WITH THE LARGEST TOTAL FORECAST ERROR
================================================================
           target_time  regions  total_actual_energy_kwh  total_predicted_energy_kwh  total_absolute_error_kwh  mean_absolute_error_kwh
2026-09-28 13:00:00+02:00       10                7932.461815                 7838.364875                113.731887                11.373189
2026-09-25 13:00:00+02:00       10                7841.354333                 7824.238787                101.224620                10.122462
2026-09-24 12:00:00+02:00       10                7788.711170                 7769.949023                 90.627973                 9.062797
2026-09-24 02:00:00+02:00       10                4545.289886                 4562.326597                 90.412046                 9.041205
2026-09-28 12:00:00+02:00       10                7850.631645                 7806.532214                 89.954144                 8.995414
2026-09-25 12:00:00+02:00       10                7791.088480                 7759.741409                 87.115449                 8.711545
2026-09-28 14:00:00+02:00       10                7751.679275                 7694.855603                 86.772961                 8.677296
2026-09-22 13:00:00+02:00       10                7785.008815                 7779.904013                 86.008320                 8.600832
2026-09-28 22:00:00+02:00       10                5167.604243                 5148.113725                 85.116801                 8.511680
2026-09-24 13:00:00+02:00       10                7826.550408                 7793.448147                 79.800196                 7.980020

================================================================
WORST-PERFORMING REGION
================================================================
Region: Region 8
MAE: 8.215959 kWh
RMSE: 11.893355 kWh
Maximum error: 49.996982 kWh
Mean percentage error: 1.023369%

================================================================
FILES SAVED
================================================================
D:\smart_energy_grid_analytics\code\forecasting\forecast_error_analysis.csv
D:\smart_energy_grid_analytics\code\forecasting\forecast_error_by_region.csv
D:\smart_energy_grid_analytics\code\forecasting\forecast_error_by_hour.csv
D:\smart_energy_grid_analytics\code\forecasting\forecast_largest_errors.csv

Forecast error analysis completed successfully.

D:\smart_energy_grid_analytics\code>
```



```
Administrator: Command Prompt
  RMSE improvement: 72.614879 (91.45%)

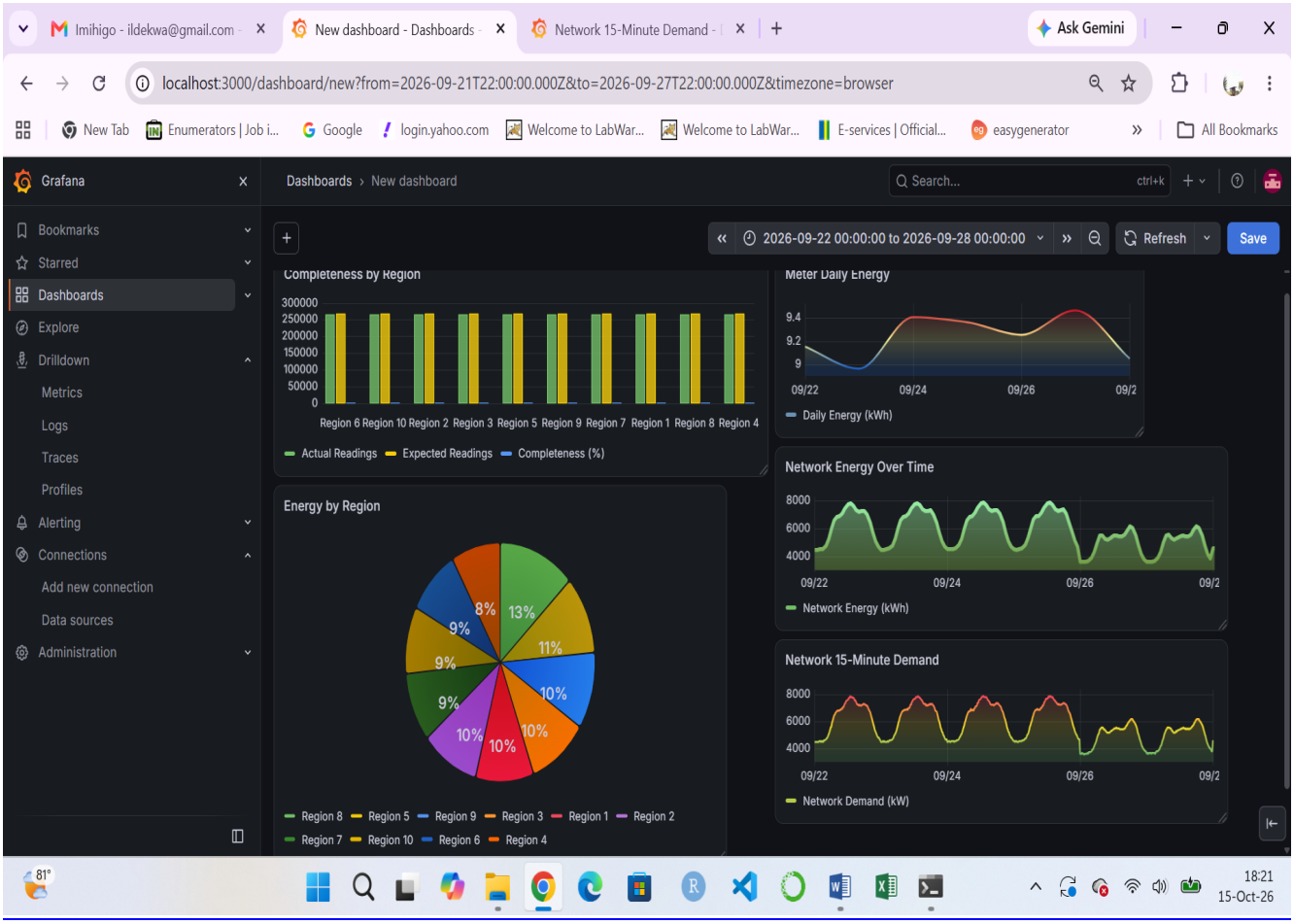
================================================================
REGIONAL COMPARISON
================================================================
   region  test_predictions  baseline_mae  baseline_rmse  model_mae  model_rmse  mae_improvement  rmse_improvement
 Region 1              168     44.873117      83.695759   5.045852    6.318021        39.827266         77.377738
Region 10              168     39.716855      74.281635   4.362955    5.667922        35.353900         68.613713
 Region 2              168     41.628612      76.810539   4.693007    5.766506        36.935605         71.044033
 Region 3              168     42.753727      77.607456   5.058712    6.244460        37.695015         71.362996
 Region 4              168     34.856463      64.232754   4.336835    5.409993        30.519628         58.822761
 Region 5              168     44.400445      81.084334   5.782263    7.457062        38.618182         73.627272
 Region 6              168     39.425942      74.766375   3.687761    4.748774        35.738181         70.017601
 Region 7              168     39.688289      73.944966   4.446307    5.604182        35.241983         68.340784
 Region 8              168     55.967231     102.057263   8.215959   11.893355        47.751272         90.163908
 Region 9              168     43.772130      80.178337   4.914869    6.016132        38.857261         74.162206

================================================================
REGIONAL PEAK-HOUR COMPARISON
================================================================
   region                 peak_time  actual_energy_kwh  baseline_prediction  baseline_absolute_error  model_prediction  model_absolute_error
 Region 1 2026-09-28 13:00:00+02:00         835.609269           565.269965               270.339304        817.050017             18.559252
Region 10 2026-09-28 13:00:00+02:00         743.034572           502.327176               240.707396        736.470826              6.563746
 Region 2 2026-09-28 13:00:00+02:00         780.125637           546.912902               233.212736        778.265757              1.859880
 Region 3 2026-09-28 13:00:00+02:00         783.000986           554.685163               228.315823        792.818459              9.817473
 Region 4 2026-09-28 18:00:00+02:00         659.902519           561.405882                98.496638        656.780212              3.122307
 Region 5 2026-09-28 13:00:00+02:00         828.231544           577.356955               250.874589        810.755294             17.476250
 Region 6 2026-09-28 13:00:00+02:00         749.111954           504.407936               244.704018        734.555961             14.555994
 Region 7 2026-09-25 13:00:00+02:00         751.049311           736.638568                14.410743        746.092389              4.956921
 Region 8 2026-09-25 13:00:00+02:00        1016.554521          1010.533746                 6.020774        966.557539             49.996982
 Region 9 2026-09-23 13:00:00+02:00         808.959570           796.886432                12.073139        803.721809              5.237761

================================================================
USEFULNESS CONCLUSION
================================================================
The Random Forest improves MAE over the previous-day baseline.
The Random Forest improves RMSE over the previous-day baseline.

================================================================
FILES SAVED
```

Dashboard


Grafana
Dashboards › New dashboard
Search...
Bookmarks
Starred
Dashboards
Explore
Drilldown
Metrics
Logs
Traces
Profiles
Alerting
Connections
Add new connection
Data sources
Administration
2026-09-22 00:00:00 to 2026-09-28 00:00:00
Refresh
Save
Completeness by Region
Region 6 Region 10 Region 2 Region 3 Region 5 Region 9 Region 7 Region 1 Region 8 Region 4
Actual Readings
Expected Readings
Completeness (%)
Meter Daily Energy
Daily Energy (kWh)
Energy by Region
Network Energy Over Time
Network Energy (kWh)
Network 15-Minute Demand
Network Demand (kW)
Region 8 Region 5 Region 9 Region 3 Region 1 Region 2
Region 7 Region 10 Region 6 Region 4
18:21
15-Oct-26